

Provably Secure and Efficient Arithmetization-Oriented Compression Functions

GAPS 2026, Vienna

Stefano Trevisani¹ joint work with E. Andreeva¹, R. Bhattacharyya², and A. Roy³

September 23, 2026

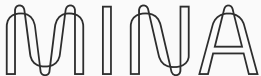
¹TU Wien, ²University of Birmingham, ³University of Innsbruck



Verifiable Computation and ZK-SNARKs

ZK-SNARKs enable many applications that need verifiable computation:

- Privacy-preserving currencies (e.g. ZCash),
- Distributed storage systems (e.g. FileCoin),
- Constant-size blockchains (e.g. Mina),
- ZK Virtual Machines (e.g. RISC Zero).

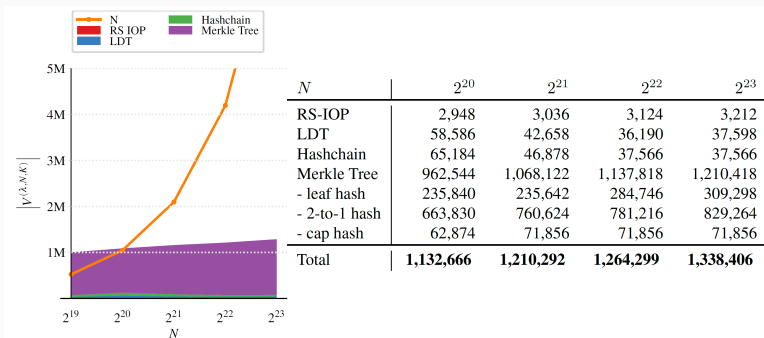


RISC
ZERO

Impact of Hashing

Compression/hash functions a key component:

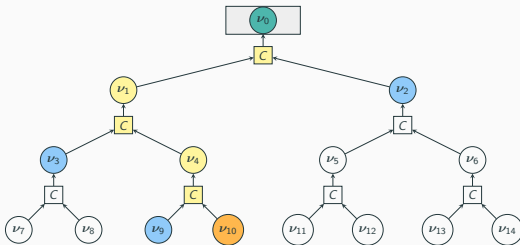
- In FRACTAL's verifier circuit [4], 90% of constraints.
- In a RISC Zero checksum program, 37% of total time.
- '*Circuit*' and *plain* performance are both important!



Cryptographic Compression functions in ZK-SNARKs

Where are compression/hash functions used?

- Fiat-Shamir for interactive $\rightarrow$ non-interactive protocols.
- **Merkle Tree VCS:**
 - Recursive proofs.
 - Post-quantum/transparent SNARKs.
 - Blockchains transactions.



Arithmetization

How are computations represented in ZK-SNARKs?

- Arithmetic circuit model over a finite field $\mathbb{F}_{p^d}$.
 - Often, p odd and 'large', and $d = 1$.
- Circuits are encoded as a set of algebraic constraints:
 - Rank-1 Constraint Systems (R1CS):

$$\left\{ ax_i \cdot by_i = cz_i \right.$$

- Plonky arithmetization:

$$\left\{ q_x x_i + q_y y_i + q_z z_i + q_{xy} x_i y_i + q_c = 0 \right.$$

- Algebraic Intermediate Representation (AIR):

$$\left\{ f_i(x_1, \dots, x_n, y_1, \dots, y_n) = 0 \right\} \wedge \left\{ B_i(x_1, \dots, x_n) = 0 \right.$$

What is an arithmetization-oriented algorithm?

- Small $\mathbb{F}_p$ -circuit guarantees efficient arithmetization.
- Equivalent representations, e.g. $x^\alpha = y \iff x = y^{1/\alpha}$.
- Some frameworks support lookup arguments.
- However, there are tradeoffs to consider.

Classic Designs are not Enough

Why not just use classical designs?

- Classical designs are bit-oriented:
 - High performance in 'plain' evaluation.
 - **Impractical when arithmetization is non-Boolean.**
 - One SHA256 compression step $\approx 10^4$ multiplication gates.
- **Arithmetization-Oriented (AO) Cryptography:**
 - Efficient arithmetization in the target field.
 - AO compression comparable to SHA256 $\approx 10^2$ multiplication gates.

We need secure arithmetization-oriented compression/hash functions!

Classical compression functions built modularly:

- Apply a mode on top of an underlying primitive.
- Blockcipher-based modes:
 - E.g. Davies-Meyer and others in the 'PGV' family (SHA2).
- Permutation-based modes:
 - E.g. Sponge functions (SHA3).

What about secure modes over an arbitrary finite field?

- For Sponge, generic security was studied in [8].

Efficient and provably secure AO compression modes?

Security Properties

Fundamental security properties for a compression mode $\mathcal{C}$:

- Ideal cipher E /ideal permutation π (both forward and inverse queries).
- Collision resistance:

$$\mathbf{Adv}_{\mathcal{C}}^{\text{COL}}(\mathcal{A}, q) = \Pr[\mathcal{C}_{\pi}(\mathbf{x}) = \mathcal{C}_{\pi}(\mathbf{x}') \wedge \mathbf{x} \neq \mathbf{x}' \mid (\mathbf{x}, \mathbf{x}') \leftarrow \mathcal{A}^{\pi}()]$$

- Preimage resistance:

$$\mathbf{Adv}_{\mathcal{C}}^{\text{PRE}}(\mathcal{A}, q) = \Pr[\mathcal{C}_{\pi}(\mathbf{x}) = \mathcal{C}_{\pi}(\mathbf{x}') \mid \mathbf{x} \xleftarrow{\$} M; \mathbf{x}' \leftarrow \mathcal{A}^{\pi}(\mathcal{C}_{\pi}(\mathbf{x}))]$$

- Random oracle indistinguishability:

$$\mathbf{Adv}_{\mathcal{C}, \mathcal{S}}^{\text{INDIF}}(\mathcal{D}, q) = \left| \Pr[\mathcal{D}^{\pi, \mathcal{C}_{\pi}}() = \top] - \Pr[\mathcal{D}^{\mathcal{S}^{\mathcal{H}}, \mathcal{H}}() = \top] \right|$$

Merkle Tree VCS Security

RO-indifferentiability is the strongest notion, but not always necessary:

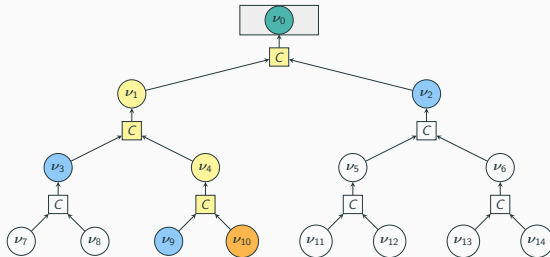
- Opening forgery resistance:

- Commitment generator $\mathcal{G}$, opening prover $\mathcal{P}$, verifier $\mathcal{V}$, ideal primitive π .

$$\mathbf{Adv}_{(\mathcal{G}, \mathcal{P}, \mathcal{V})}^{\text{OPEN}}(\mathcal{A}, q) = \Pr[\tilde{\omega} \neq \mathcal{P}(\mathbf{m}, i) \wedge \mathcal{V}(\mathcal{G}(\mathbf{m}), \tilde{\omega}, i) = \top \mid (\mathbf{m}, \tilde{\omega}, i) \leftarrow \mathcal{A}^\pi()]$$

- For a Merkle Tree VCS, it holds that:

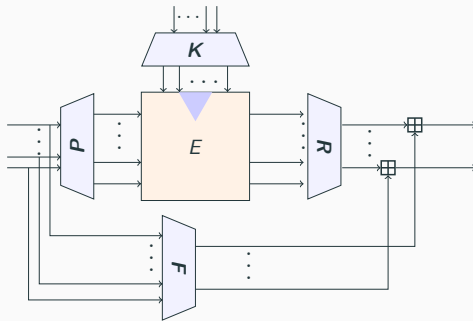
$$\mathbf{Adv}_{(\mathcal{G}, \mathcal{P}, \mathcal{V})}^{\text{OPEN}}(q) \leq \mathbf{Adv}_C^{\text{COL}}(q)$$



The PGV-ELC modes

At CSF'24, we introduced the PGV-ELC AO compression modes:

- Underlying primitive is a block cipher.
- Generalize and extend the classical PGV family.
- Expanding transformations $\mathbf{K} \in \mathbb{F}_p^{\kappa \times \kappa'}$ and $\mathbf{P} \in \mathbb{F}_p^{n \times n'}$.
- Compressing transformations $\mathbf{F} \in \mathbb{F}_p^{\ell \times n'}$ and $\mathbf{G} \in \mathbb{F}_p^{\ell \times n}$.



Collision resistance of PGV-ELC:

1. Consider $\mathbf{K}$ and $\mathbf{P}$ left-invertible (full column rank), $\mathbf{F}$ and $\mathbf{R}$ right-invertible (full row rank).
 - Induce partitions of their output (or input) spaces.
2. Adversary can adaptively exploit partition unbalances due to oracle replies.
3. 'Meaningless' queries, can be exploited only indirectly.
4. Overall, $\mathbf{Adv}_C^{\text{COL}}(q) \leq \frac{q^2+q}{p^\ell-q}$.
5. Similarly for preimage resistance: $\mathbf{Adv}_C^{\text{PRE}}(q) \leq \frac{q}{p^\ell-q}$.

PGV-ELC Proof Generation

Time to generate a MT opening proof in Groth16 (libsnark):

- Scalar field of the BLS12-381 elliptic curve: $\log_2(p) \approx 255$.
- Target ≈ 128 bits of security.

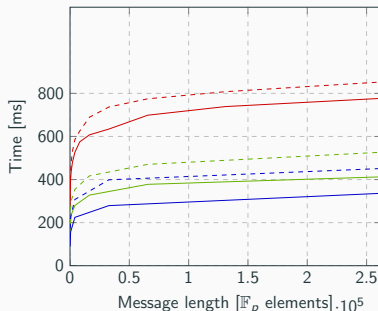


Figure 1: Dashed: POSEIDON, Solid: HADES in the PGV-ELC mode.

PGV-ELC Root Computation

Time to compute the root of a Merkle Tree (`libff`):

- Scalar field of the BLS12-381 elliptic curve: $\log_2(p) \approx 255$.
- Target ≈ 128 bits of security.

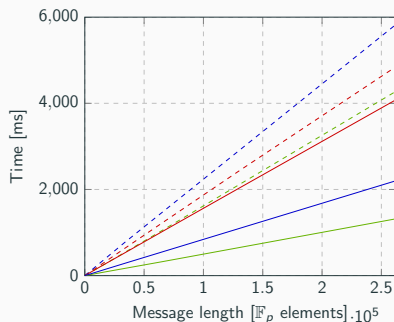


Figure 2: Dashed: POSEIDON, Solid: HADES in the PGV-ELC mode.

Limitations of AO blockciphers

Blockciphers in the AO setting have received relatively little attention:

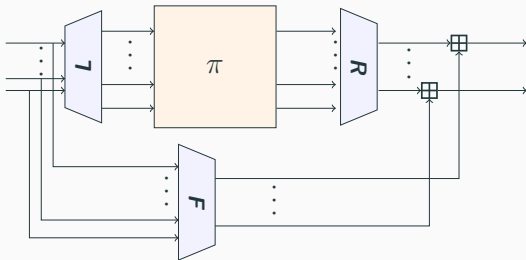
- Security of the standard parametrization of the underlying primitive is not obvious.
- Most AO cryptanalysis literature focuses on (unkeyed) permutations.
- Typically, compression functions are single-iteration sponges.
- Dedicated compression modes have also been proposed: Jive, Trunc.
 - Higher compression efficiency.
 - Lacked a formal security proof.

Efficient and provably secure permutation-based AO compression modes?

The PAX Family

At USENIX'26, we introduced the PAX AO compression modes:

- Underlying primitive is a permutation.
- Expanding transformation $\mathbf{L} \in \mathbb{F}_p^{n \times m}$.
- Compressing transformations $\mathbf{F} \in \mathbb{F}_p^{\ell \times m}$ and $\mathbf{R} \in \mathbb{F}_p^{\ell \times n}$.



PAX as a Generalization of Other AO Compression Functions

The PAX family captures and generalizes other m -to- ℓ compression functions:

- $\text{Trunc}(\mathbf{x}) = \lfloor \pi(\mathbf{x}) + \mathbf{x} \rfloor_\ell$:
 - $n = m$, $\mathbf{L}, \mathbf{F}, \mathbf{R}$ are (rectangular) identities.
 - $\text{Jive}(\mathbf{x}) = \sum_i \mathbf{x}_i + \pi(\mathbf{x})_i$:
 - $n = m$, $\mathbf{L}$ is the identity, $\mathbf{F}$ and $\mathbf{R}$ are block diagonal.
 - $\text{Sponge-1}(\mathbf{x}) = \lfloor \pi(\mathbf{x} \parallel \mathbf{0}^c) \rfloor_\ell$, for some $c \geq 0$.
 - $n = m + c$, $\mathbf{L}$ and $\mathbf{R}$ are (rectangular) identities, $\mathbf{R}$ is the zero matrix.
- ✓ A unified framework to study the security of a larger class of modes.

Indifferentiability of PAX

RO-indifferentiability based on the following simulator:

```
function  $\mathcal{S}^{\mathcal{H}}(b \in \{-1, 1\}, \mathbf{z} \in \mathbb{F}_p^n) \rightarrow \mathbb{F}_p^n$ 
```

```
  static  $T \leftarrow \emptyset$ 
```

```
  if  $b = 1$  then
```

▷ Forward query

```
    if  $\mathbf{z} \in \text{Dom}(T)$  then return  $T(\mathbf{z})$ 
```

```
     $\mathbf{v} \leftarrow L^+ \mathbf{z}$ 
```

```
    if  $\mathbf{z} = L\mathbf{v}$  then
```

▷ Queried point has a preimage

```
       $\mathbf{h} \leftarrow \mathcal{H}(\mathbf{v})$ 
```

```
       $\mathbf{y} \xleftarrow{\$} [\mathbf{h} - F\mathbf{v}]_R \setminus \text{Ran}(T)$ 
```

▷ $[\mathbf{x}]_R$ equiv. class of $\mathbf{x}$

```
    else
```

▷ Queried point is preimage-free

```
       $\mathbf{y} \xleftarrow{\$} \mathbb{F}_p^n \setminus \text{Ran}(T)$ 
```

```
       $T \leftarrow T \cup \{(\mathbf{z}, \mathbf{y})\}$ 
```

```
    return  $\mathbf{y}$ 
```

```
  else
```

▷ Backward query

```
    if  $\mathbf{z} \in \text{Ran}(T)$  then return  $T^{-1}(\mathbf{z})$ 
```

```
     $\mathbf{x} \xleftarrow{\$} \mathbb{F}_p^n \setminus \text{Dom}(T)$ 
```

```
     $T \leftarrow T \cup \{(\mathbf{x}, \mathbf{z})\}$ 
```

```
  return  $\mathbf{x}$ 
```

Security Comparison

Mode	Primitive	COL	PRE	INDIF
Random Oracle	$\text{Func}(m, \ell)$	q^2/p^ℓ	q/p^ℓ	0
PAX	$\text{Perm}(m + c)$	q^2/p^ℓ	q/p^ℓ	q/p^c
Sponge-1	$\text{Perm}(m + c)$	$\min\{q^2/p^\ell, q/p^c\}$	$q/p^{\min\{\ell, c\}}$	q/p^c
Sponge	$\text{Perm}(r + c)$	$q^2/p^{\min\{\ell, c\}}$	$q/p^{\min\{\ell, c\}}$	q^2/p^c

- PAX bounds conditional on the left/right-invertibility of $\mathbf{L}, \mathbf{F}, \mathbf{R}$.
- PAX offers optimal COL and PRE resistance.
- Trunc, Jive satisfy conditions and follow the bound (with $c = 0$).
- For Sponge-1, $\mathbf{R} = \mathbf{0}$ not invertible, hence deviation.

Parametrization of PAX modes

128-bit COL		$\log_2(p) \approx 256$		$\log_2(p) \approx 64$	
Comp. ratio	Mode	(m, ℓ)	n	(m, ℓ)	n
2:1	PAX	(2, 1)	2	(8, 4)	8
	Sponge-1	(2, 1)	3	(8, 4)	12
4:1	PAX	(4, 1)	4	(16, 4)	16
	Sponge-1	(4, 1)	5	(16, 4)	20
8:1	PAX	(8, 1)	8	(32, 4)	32
	Sponge-1	(8, 1)	9	(32, 4)	36

- For a given security level, PAX modes need a smaller permutation.
- In the ideal model, only left/right-invertibility of $\mathbf{L}$, $\mathbf{R}$, $\mathbf{F}$ needed
 - (Rectangular) identities are in principle sufficient.
 - Lift the ‘final MDS’ design practice, clear separation of concerns.

Experiments setup:

- 128-bit security level (collision resistance).
- Two popular AO permutations: POSEIDON- π and *Rescue Prime*.
- Plain evaluation performance in software (x86-64).
- Two different arithmetization techniques:
 - R1CS, used in implementations of Groth16:
 - Plonkyish, used e.g. in Plonky2:

Plain performance

Compression latency (x86-64, Raptor Cove @ 3.0GHz):

Comp. ratio	BN254			Goldilocks		
	PAX	Sponge-1	Speed-up	PAX	Sponge-1	Speed-up
POSEIDON- π						
2 : 1	9.4 μs	17.5 μ s	1.86 $\times$	3.5 μs	7.3 μ s	2.09 $\times$
4 : 1	28.2 μs	42.4 μ s	1.50 $\times$	12.2 μs	18.6 μ s	1.52 $\times$
8 : 1	103.7 μs	130.4 μ s	1.26 $\times$	141.5 μs	265.4 μ s	1.88 $\times$
Rescue						
2 : 1	334.6 μs	353.2 μ s	1.06 $\times$	16.8 μs	26.2 μ s	1.56 $\times$
4 : 1	372.0 μs	382.9 μ s	1.03 $\times$	36.2 μs	46.9 μ s	1.30 $\times$
8 : 1	553.5 μs	626.1 μ s	1.13 $\times$	117.7 μs	153.5 μ s	1.30 $\times$

Groth16 benchmarks

Preimage circuit in R1CS (Groth16/libsnark):

Comp. ratio	# constraints		Proof Generation time		
	PAX	Sponge-1	PAX	Sponge-1	Speed-up
POSEIDON- π					
2 : 1	221	246	44.5 ms	47.3 ms	1.06×
4 : 1	268	293	51.3 ms	54.9 ms	1.07×
8 : 1	368	393	66.3 ms	71.5 ms	1.08×
<i>Rescue</i>					
2 : 1	240	252	41.2 ms	42.3 ms	1.03×
4 : 1	264	270	44.9 ms	45.1 ms	1.00×
8 : 1	384	432	63.8 ms	67.7 ms	1.06×

Preimage circuit in $\mathcal{PlonK}$ (*Plonky2*):

Comp. ratio	# gates		Proof Generation time		
	PAX	Sponge-1	PAX	Sponge-1	Speed-up
POSEIDON- π					
2 : 1	122	259	11.3 ms	16.5 ms	1.46×
4 : 1	439	668	26.0 ms	27.1 ms	1.04×
8 : 1	2065	2864	90.8 ms	92.9 ms	1.02×
<i>Rescue</i>					
2 : 1	91	175	10.9 ms	17.2 ms	1.58×
4 : 1	284	418	16.8 ms	27.1 ms	1.61×
8 : 1	976	1213	47.9 ms	49.0 ms	1.02×

Fin

Thank you for your attention!

-  Elena Andreeva, Rishiraj Bhattacharyya, Arnab Roy, and Stefano Trevisani.
**On Efficient and Secure Compression Functions for
Arithmetization-Oriented Hashing.**
In *2024 IEEE 37th Computer Security Foundations Symposium (CSF)*, pages
1–16, Los Alamitos, CA, USA, Jul 2024. IEEE Computer Society.
-  Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche.
On the indifferentiability of the sponge construction.
In Nigel Smart, editor, *Advances in Cryptology – EUROCRYPT 2008*, pages
181–197, Berlin, Heidelberg, 2008. Springer Berlin Heidelberg.



Clémence Bouvier, Pierre Briaud, Pyrros Chaidos, Léo Perrin, Robin Salen, Vesselin Velichkov, and Danny Willems.

New design techniques for efficient arithmetization-oriented hash functions: Anemoi permutations and Jive compression mode.

In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023*, pages 507–539, Cham, 2023. Springer Nature Switzerland.



Alessandro Chiesa, Dev Ojha, and Nicholas Spooner.

Fractal: Post-quantum and transparent recursive proofs from holography.

In Anne Canteaut and Yuval Ishai, editors, *Advances in Cryptology – EUROCRYPT 2020*, pages 769–793, Cham, 2020. Springer International Publishing.

 Lorenzo Grassi, Dmitry Khovratovich, Christian Rechberger, Arnab Roy, and Markus Schofnegger.

Poseidon: A new hash function for Zero-Knowledge proof systems.

In *30th USENIX Security Symposium (USENIX Security 21)*, pages 519–535.
USENIX Association, aug 2021.

 Lorenzo Grassi, Dmitry Khovratovich, and Markus Schofnegger.

Poseidon2: A faster version of the poseidon hash function.

Cryptology ePrint Archive, Paper 2023/323, 2023.



Jens Groth.

Short non-interactive zero-knowledge proofs.

In *Advances in Cryptology - ASIACRYPT 2010 - 16th International Conference on the Theory and Application of Cryptology and Information Security*, volume 6477 of *Lecture Notes in Computer Science*, pages 341–358. Springer, 2010.



Dmitry Khovratovich, Mario Marhuenda Beltrán, and Bart Mennink.

Generic security of the safe api and its applications.

In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology – ASIACRYPT 2023*, pages 301–327, Singapore, 2023. Springer Nature Singapore.



Ralph Charles Merkle.

Secrecy, Authentication, and Public Key Systems.

PhD thesis, Stanford University, Stanford, CA, USA, 1979.

AAI8001972.



Bart Preneel, René Govaerts, and Joos Vandewalle.

Hash functions based on block ciphers: A synthetic approach.

In *Advances in Cryptology - CRYPTO '93, 13th Annual International Cryptology Conference, Santa Barbara, California, USA, August 22-26, 1993, Proceedings*, volume 773 of *Lecture Notes in Computer Science*, pages 368–378. Springer, 1993.

-  Tomer Solberg.
Risc zero prover protocol & analysis.
https://github.com/ingonyama-zk/papers/blob/main/risc0_protocol_analysis.pdf, 04 2023.
Accessed: 2026-02-08.